

1.1. ESTRUCTURA Y FUNCIONAMIENTO DEL COMPUTADOR

El computador es una máquina destinada a procesar datos. En una visión esquemática, como la que muestra la figura 1.1, este procesamiento involucra dos flujos de información: el de datos y el de instrucciones. Se parte del flujo de datos que han de ser procesados. Este flujo de datos es tratado mediante un flujo de instrucciones máquina, generado por la ejecución de un programa, produciendo el flujo de datos resultado.

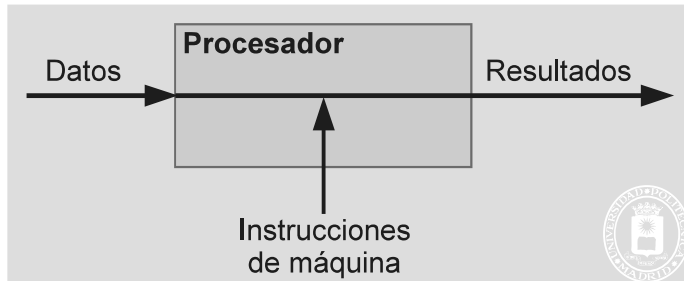


Figura 1.1 Esquema de funcionamiento del computador.

Para llevar a cabo la función de procesamiento, un computador con arquitectura von Neumann está compuesto por los cuatro componentes básicos representados en la figura 1.2.

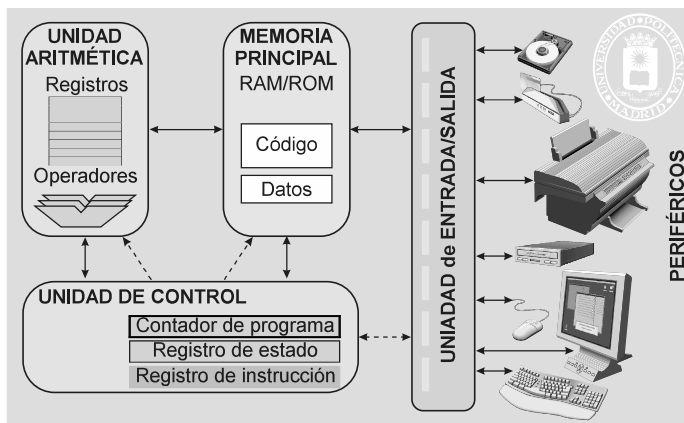


Figura 1.2 Componentes básicos del computador con arquitectura von Neumann: Memoria principal, unidad aritmética, unidad de control y unidades de entrada/salida.

Se denomina **procesador** o unidad central de proceso (**UCP**) al conjunto de la unidad aritmética y de la unidad de control. Actualmente, en un único circuito integrado se puede incluir varios procesadores (llamados núcleos o *cores*), además de la unidad de gestión de memoria que se describe en la sección “4.2.7 Concepto de memoria virtual y memoria real”.

Desde el punto de vista de los sistemas operativos, nos interesa más profundizar en el funcionamiento interno del computador que en los componentes físicos que lo constituyen.

Memoria principal

La **memoria principal** se construye con memoria RAM (*Random Access Memory*) y memoria ROM (*Read Only Memory*). En ella han de residir los datos a procesar, el programa máquina a ejecutar y los resultados (aclaración 1.1). La memoria está formada por un conjunto de celdas idénticas. Mediante la información de dirección se selecciona de forma única la celda sobre la que se quiere realizar el acceso, pudiendo ser éste de lectura o de escritura. En los computadores actuales es muy frecuente que el direccionamiento se realice a nivel de byte, es decir, que las direcciones 0, 1, 2,... identifiquen los bytes 0, 1, 2,... de memoria. Sin embargo, como se muestra en la figura 1.3, el acceso se realiza generalmente sobre una **palabra** de varios bytes (típicamente de 4 o de 8 bytes) cuyo primer byte se sitúa en la dirección utilizada (que, por tanto, tiene que ser múltiplo de 4 o de 8).

Aclaración 1.1. Se denomina programa máquina (o código) al conjunto de instrucciones máquina que tiene por objeto que el computador realice una determinada función. Los programas escritos en cualquiera de los lenguajes de programación han de convertirse en programas máquina para poder ser ejecutados por el computador.

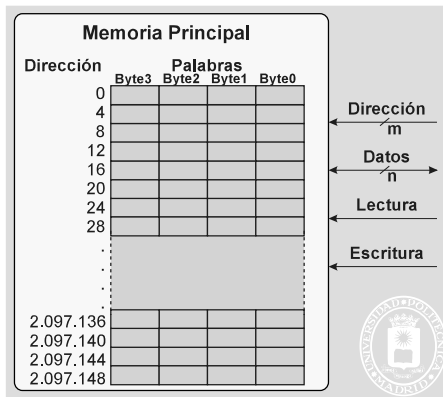


Figura 1.3 La Unidad de memoria está compuesta por un conjunto de celdas iguales que se seleccionan mediante una dirección. Las señales de control de lectura y escritura determinan la operación a realizar.

Unidad aritmético-lógica

La **unidad aritmético-lógica** permite realizar una serie de operaciones aritméticas y lógicas sobre uno o dos operandos. Como muestra la figura 1.4, los datos sobre los que opera esta unidad están almacenados en un conjunto de registros o bien provienen directamente de la memoria principal. Por su lado, los resultados también se almacenan en registros o en la memoria principal.

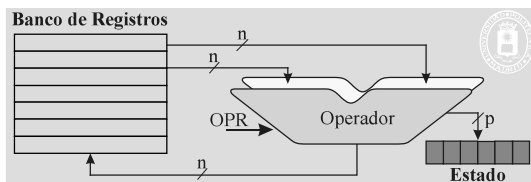


Figura 1.4 Estructura de la unidad aritmético-lógica. Pueden observarse el banco de registros, los operadores y el registro de estado.

Otro aspecto muy importante de la unidad aritmética es que, además del resultado de la operación solicitada, genera unos resultados adicionales que se cargan en el registro de estado del computador. Típicamente dichos resultados adicionales son los siguientes:

- ◆ **Cero:** Se pone a “1” si el resultado es cero.
- ◆ **Signo:** Se pone a “1” si el resultado es negativo.
- ◆ **Acarreo:** Se pone a “1” si el resultado tiene acarreo.
- ◆ **Desbordamiento:** Se pone a “1” si el resultado tiene desbordamiento.

La importancia de estos bits de estado es que las instrucciones de salto condicional se realizan sobre ellos, por lo que son los bits que sirven para tomar decisiones en los programas.

Unidad de control

La **unidad de control** es la que se encarga de hacer funcionar al conjunto, para lo cual realiza cíclicamente la siguiente secuencia:

- Lee de memoria la siguiente instrucción máquina que forma el programa.
- Interpreta la instrucción leída: aritmética, lógica, de salto, etc.
- Lee, si los hay, los datos de memoria referenciados por la instrucción.
- Ejecuta la instrucción.
- Almacena, si lo hay, el resultado de la instrucción.

La unidad de control tiene asociados una serie de registros, entre los que cabe destacar: el **contador de programa (PC, Program Counter)**, que indica la dirección de la siguiente instrucción máquina a ejecutar; el **puntero de pila (SP, Stack Pointer)**, que sirve para manejar cómodamente una pila en memoria principal; el **registro de instrucción (RI)**, que permite almacenar —una vez leída de la memoria principal— la instrucción máquina a ejecutar, y el **registro de estado (RE)**, que almacena diversa información producida por la ejecución de alguna de las últimas instrucciones del programa (bits de estado aritméticos) e información sobre la forma en que ha de comportarse el computador (bits de interrupción, modo de ejecución, etc.).

Unidad de entrada/salida

Finalmente, la **unidad de entrada/salida (E/S)** se encarga de hacer la transferencia de información entre la memoria principal (o los registros generales) y los periféricos. La entrada/salida se puede hacer bajo el gobierno de la unidad de control (E/S programada) o de forma independiente (acceso directo a memoria o DMA), como se verá en la sección “1.6 Entrada/Salida”.

Generalmente, el modo usuario no permite operaciones de E/S, ni modificar una gran parte del registro de estado, ni modificar los registros de soporte de gestión de memoria. La figura 1.6 muestra un ejemplo de dos modelos de programación de un computador. También es frecuente que el procesador incluya dos punteros de pila (SP y SP'), el SP para usarlo en modo usuario y el SP' para usarlo en modo privilegiado.

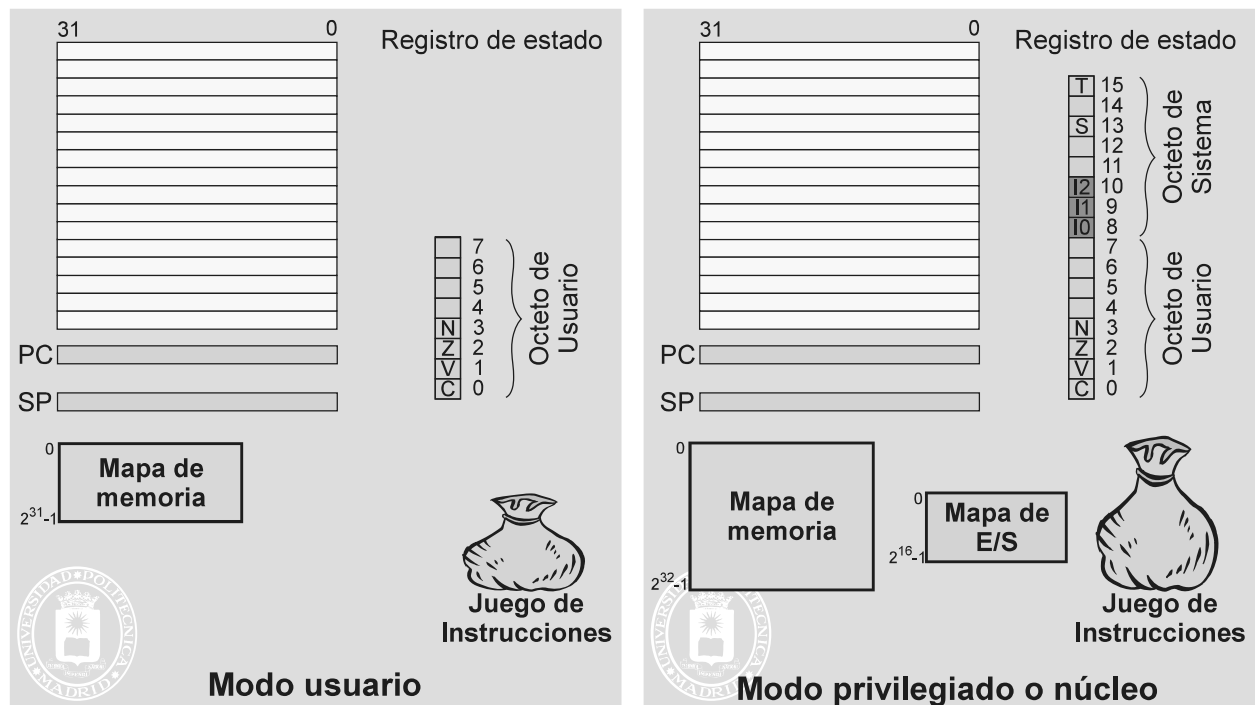


Figura 1.6 Modelos de programación de usuario y de privilegiado.

1.2.2. Secuencia de funcionamiento del procesador

La unidad de control del procesador es la que establece el funcionamiento del mismo. Este funcionamiento está basado en una secuencia sencilla, que se repite sin cesar y a muy alta velocidad (miles de millones de veces por segundo). Como muestra la figura 1.7, esta secuencia consiste en tres pasos: a) lectura de memoria principal de la instrucción máquina apuntada por el contador de programa, b) incremento del contador de programa —para que apunte a la siguiente instrucción máquina— y c) ejecución de la instrucción (que puede incluir la lectura de operandos en memoria o el almacenamiento de resultados en memoria). Esta secuencia tiene dos propiedades fundamentales: es lineal, es decir, ejecuta de forma consecutiva las instrucciones que están en direcciones consecutivas, y forma un bucle infinito. Esto significa que la unidad de control del procesador está continua e ininterrumpidamente realizando esta secuencia (advertencia 1.1).

Advertencia 1.1. Algunos procesadores tienen una instrucción de parada (p. ej.: HALT) que hace que la unidad de control se detenga hasta que llegue una interrupción. Sin embargo, esta instrucción es muy poco utilizada (salvo en equipos portátiles en los que interesa ahorrar batería), por lo que, a efectos prácticos, podemos considerar que la unidad de control no para nunca de realizar la secuencia de lectura de instrucción, incremento de PC y ejecución de la instrucción.

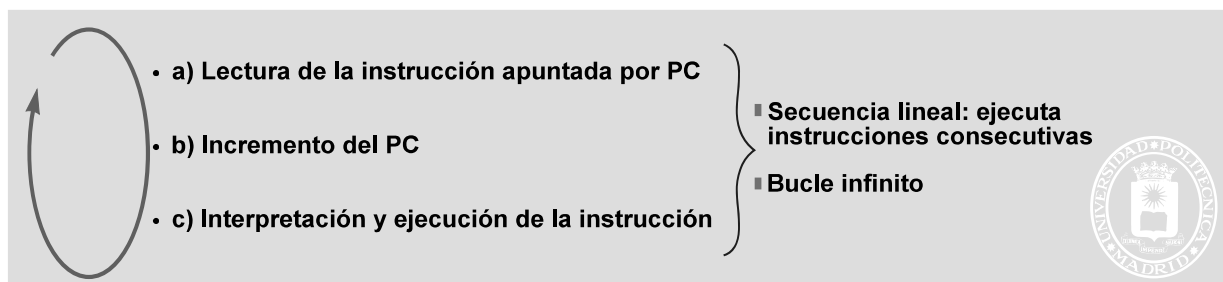


Figura 1.7 Secuencia de ejecución del procesador.

Podemos decir, por tanto, que lo único que sabe hacer el procesador es repetir a gran velocidad esta secuencia. Esto quiere decir que, para que realice algo útil, se ha de tener, adecuadamente cargado en memoria, un programa máquina con sus datos, y se ha de conseguir que el contador de programa apunte a la instrucción máquina inicial de dicho programa.

14 Sistemas operativos

El esquema de ejecución lineal es muy limitado, por lo que se añaden unos mecanismos que permiten alterar esta ejecución lineal. Todos ellos se basan en algo muy simple: modifican el contenido del contador de programa, con lo que se consigue que se salte o bifurque a otra sección del programa o a otro programa (que, lógicamente, también ha de residir en memoria).

Los tres mecanismos básicos de ruptura de secuencia son los siguientes:

- Las instrucciones máquina de salto o bifurcación, que permiten que el programa rompa su secuencia lineal de ejecución, pasando a otra sección de sí mismo.
- Las interrupciones externas o internas, que hacen que la unidad de control modifique el valor del contador de programa, saltando a otro programa (que deberá ser el sistema operativo).
- Una instrucción máquina de llamada al sistema (p. ej.: TRAP, INT o SC), que produce un efecto similar a la interrupción, haciendo que se salte a otro programa (que deberá ser el sistema operativo).

Si desde el punto de vista de la programación el interés se centra en las instrucciones de salto, y, en especial en las de salto a procedimiento y retorno de procedimiento, desde el punto de vista de los sistemas operativos son mucho más importantes las interrupciones y las instrucciones máquina de llamada al sistema. Por tanto, centraremos nuestro interés en resaltar los aspectos fundamentales de estas dos últimas.

1.2.3. Registros de control y estado

Como se ha indicado anteriormente, la unidad de control tiene asociada una serie de registros que denominamos de control y estado. Estos registros dependen de la arquitectura del procesador. Muchos de ellos se refieren a aspectos que se analizarán a lo largo del texto, por lo que no se intentará explicar aquí su función. Entre los más importantes se pueden encontrar los siguientes:

- Contador de programa PC. Contiene la dirección de la siguiente instrucción máquina.
- Puntero de pila SP. Contiene la dirección de la cima de la pila. En algunos procesadores existen dos punteros de pila: uno para la pila del sistema operativo y otra para la del usuario.
- Registro de instrucción RI. Contienen la instrucción en curso de ejecución.
- Registro de estado, que contiene, entre otros, los bits siguientes:
 - ◆ Bits de estado aritméticos como: Signo, Acarreo, Cero y Desbordamiento.
 - ◆ Bits de modo de ejecución. Indican el modo en el que ejecuta el procesador.
 - ◆ Bits de control de interrupciones. Establecen las interrupciones que se pueden aceptar.
- Registros de gestión de memoria, como pueden ser los registros de protección de memoria o el registro identificador del espacio de direccionamiento (véase la sección “1.7.2 Mecanismos de protección de memoria”).

Algunos de estos registros son **visibles** en el modo de ejecución de usuario, como el PC, el SP y parte del estado, pero otros no lo son, como los de gestión de memoria.

Al contenido de todos los registros del procesador en un instante determinado le denominamos **estado del procesador**, término que utilizaremos profusamente a lo largo del libro. Un subconjunto del estado del procesador lo constituye el **estado visible** del procesador, formado por el conjunto de los registros visibles en modo usuario.

1.3. INTERRUPTACIONES

Una interrupción se solicita activando una señal que llega a la unidad de control. El agente generador o solicitante de la interrupción ha de activar la mencionada señal cuando necesite que se le atienda, es decir, que se ejecute un programa que le atienda.

Ante la solicitud de una interrupción, siempre y cuando esté habilitada ese tipo de interrupción, la unidad de control realiza un **ciclo de aceptación de interrupción**. Este ciclo se lleva a cabo en cuanto termina la ejecución de la instrucción máquina que se esté ejecutando, y los pasos que realiza la unidad de control son los siguientes:

- **Salva algunos registros** del procesador, como son el de estado y el contador de programa. Normalmente utiliza para ello la **pila de sistema**, gestionada por el puntero de pila SP'.
- **Eleva el modo de ejecución** del procesador, pasándolo a núcleo.
- **Carga un nuevo valor en el contador de programa**, por lo que se pasa a ejecutar otro programa, que, generalmente, será el sistema operativo.
- En muchos procesadores **inhibe** las interrupciones (véase más adelante “Niveles de Interrupción”).

La figura 1.8 muestra la interrupción vectorizada, solución usualmente utilizada para determinar la dirección de salto. Se puede observar que el agente que interrumpe suministra el llamado **vector de interrupción** que determina la dirección de comienzo del programa que desea que le atienda (programa que se suele denominar rutina de tratamiento de interrupción). La unidad de control, utilizando un direccionamiento indirecto, toma la mencionada dirección de una tabla de interrupciones IDT (*Interrupt Descriptor Table*) y la carga en el contador de programa. El resultado de esta carga es que la siguiente instrucción máquina ejecutada es la primera del mencionado programa de tratamiento de interrupción.

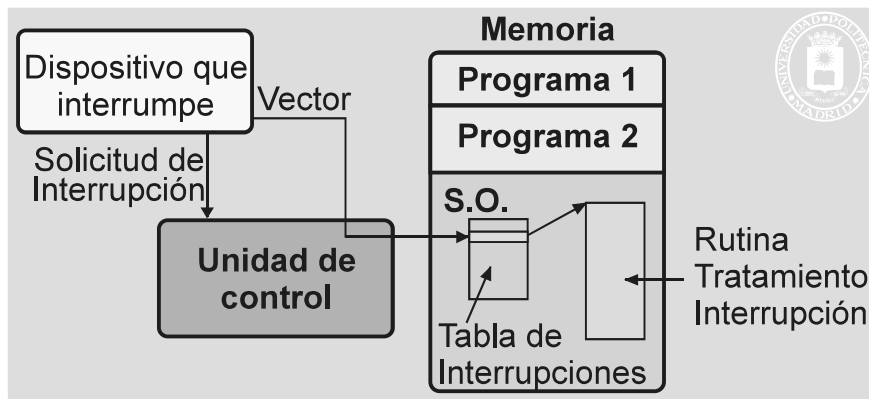


Figura 1.8 Acceso a la rutina de tratamiento de la interrupción.

Observe que se han incluido como parte del sistema operativo tanto la tabla IDT como la rutina de tratamiento de la interrupción. Esto debe ser así por seguridad, ya que la rutina de tratamiento de interrupción ejecuta en modo privilegiado. En caso contrario, un programa de usuario ejecutaría sin limitación ninguna, por lo que podría acceder a los datos y programas de otros usuarios. Como se verá más adelante, la seguridad es una de las funciones primordiales del sistema operativo.

Se dice que la interrupción es **síncrona** cuando es consecuencia directa de las instrucciones máquina que se están ejecutando. En el resto de los casos se dice que es **asíncrona**. Las interrupciones se pueden generar por diversas causas, que clasificaremos de la siguiente forma:

- **Excepciones *hardware* síncronas.** Hay determinadas causas que hacen que un programa presente una incidencia en su ejecución, por lo que se generará una interrupción, para que el sistema operativo entre a ejecutar y decida lo que debe hacerse. Dichas causas se pueden estructurar en las tres clases siguientes:
 - ◆ Problemas de ejecución:
 - Operación inválida en la unidad aritmética.
 - División por cero.
 - Operando no normalizado.
 - Desbordamiento en el resultado, siendo demasiado grande o demasiado pequeño.
 - Resultado inexacto en la unidad aritmética.
 - Dispositivo no existente (p. ej.: no existe coprocesador).
 - Región de memoria inválida.
 - Región de memoria privilegiada en modo de ejecución usuario.
 - Desbordamiento de la pila.
 - Violación de los límites de memoria asignada.
 - Error de alineación en acceso a memoria.
 - Código de operación máquina inválido.
 - Código de operación máquina privilegiado en modo de ejecución usuario.
 - ◆ Depuración:
 - Punto de ruptura.
 - ◆ Fallo de página.

Todas ellas son producidas directa o indirectamente por el programa en ejecución, por lo que decimos que se trata de interrupciones **síncronas** (advertencia 1.2).

- **Excepciones *hardware* asíncronas.** Se trata de interrupciones **asíncronas** producidas por un error en el *hardware*. En muchos textos se denominan simplemente excepciones *hardware*. Ejemplos son los siguientes:
 - ◆ Error de paridad en bus.
 - ◆ Error de paridad en memoria.
 - ◆ Fallo de alimentación.
 - ◆ Límite de temperatura excedido.

En las excepciones *hardware*, tanto síncronas como asíncronas, será el módulo del computador que produce la excepción el que genere el vector de interrupción. Además, dicho módulo suele cargar en un registro o en la pila un código que especifica el tipo de problema encontrado.

- **Interrupciones externas.** Se trata de interrupciones **asíncronas** producidas por elementos externos al procesador como son: a) el reloj, que se analizará en detalle en la sección siguiente, b) los controladores de los dispositivos de E/S, que necesitan interrumpir para indicar que han terminado una operación o conjunto de ellas, o que necesitan atención, y c) otros procesadores.
- Instrucciones máquina de **llamada al sistema** (p. ej.: `TRAP`, `INT`, `SYSENTER` o `SC`). Estas instrucciones permiten que un programa genere una interrupción de tipo síncrono. Como veremos más adelante,

estas instrucciones se incluyen para que los programas de usuario puedan solicitar los servicios del sistema operativo.

Advertencia 1.2. Las **excepciones hardware síncronas** se denominan muy frecuentemente **excepciones software**, pero en este texto reservamos dicho nombre para el concepto de excepción soportado por el sistema operativo (véase sección “3.5 Señales y excepciones”, página 83).

Como complemento al mecanismo de aceptación de interrupción, los procesadores incluyen una instrucción máquina para retornar desde la rutina de tratamiento de interrupción (p. ej.: RETI). El efecto de esta instrucción es restituir los registros de estado y PC, desde el lugar en que fueron salvados al aceptarse la interrupción (p. ej.: desde la pila del sistema).

Niveles de interrupción

Como muestra la figura 1.9, los procesadores suelen incluir varias líneas de solicitud de interrupción, cada una de las cuales puede tener asignada una determinada prioridad. En caso de activarse al tiempo varias de estas líneas, se tratará la de mayor prioridad, quedando las demás a la espera de ser atendidas. Las más prioritarias suelen ser las excepciones *hardware* asíncronas, seguidas por las excepciones *hardware* síncronas (o de programa), las interrupciones externas y las de llamada al sistema o TRAP.

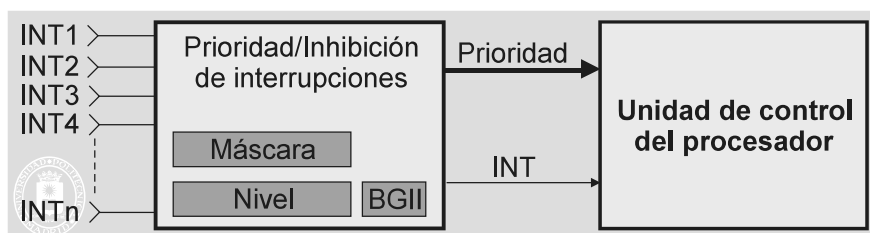


Figura 1.9 Mecanismo de inhibición/prioridad de interrupción.

Además, el procesador suele incluir un mecanismo de **inhibición** selectiva que permite detener todas o determinadas líneas de interrupción. Este mecanismo, que es muy específico de cada máquina, puede estar basado en los dos registros de máscara y de nivel, y en el biestable general de inhibición de interrupción (BGII). El comportamiento de estos mecanismos es el siguiente. Mientras esté activo BGII no se admitirá ninguna interrupción. El registro de **nivel** inhibe todas las interrupciones con prioridad menor o igual que el valor que contenga. Finalmente, el registro de **máscara** tiene un bit por línea de interrupción, por lo que permite inhibir de forma selectiva cualquiera de las líneas de interrupción.

Las interrupciones de las líneas inhibidas no son atendidas hasta que, al modificarse los valores de máscara, nivel o BGII, dejen de estar inhibidas. Dependiendo del módulo que las genera y del *hardware* de interrupción, se pueden encolar o se pueden llegar a perder algunas de las interrupciones inhibidas. Estos valores de máscara, nivel y BGII deben incluirse en la parte del registro de estado que solamente es modificable en modo privilegiado, por lo que su modificación queda restringida al sistema operativo. La unidad de control, al aceptar una interrupción, suele modificar determinados valores de los registros de inhibición para facilitar el tratamiento de las mismas. Las alternativas más empleadas son dos: inhibir todas las interrupciones o inhibir las interrupciones que tengan prioridad igual o menor que la aceptada.

Tratamiento de interrupciones

En el ciclo de aceptación de una interrupción se salvan algunos registros, pero un correcto tratamiento de las interrupciones exige preservar los valores del resto de los registros del programa interrumpido. De esta forma, se podrán restituir posteriormente dichos valores, para continuar con la ejecución del programa como si no hubiera pasado nada. Téngase en cuenta que el nuevo programa, que pone en ejecución la interrupción, cargará nuevos valores en los registros del procesador. También es necesario mantener la información que tenga el programa en memoria, para lo cual basta con no modificar la zona de memoria asignada al mismo.

Además, la rutina de tratamiento de interrupciones deberá, en cuanto ello sea posible, restituir los valores de los registros de inhibición de interrupciones para admitir el mayor número posible de interrupciones.

La instrucción máquina de retorno de interrupción RETI realiza una función inversa a la del ciclo de aceptación de la interrupción. Típicamente, restituye el valor del registro de estado E y el del contador de programa PC. En muchos casos, la instrucción RETI producirá, de forma indirecta, el paso del procesador a modo usuario. En efecto, supóngase que está ejecutando el programa A en modo usuario y que llega una interrupción. El ciclo de aceptación salva el registro de estado (que tendrá un valor Ea) y el contador de programa (que tendrá un valor PCa). Como el bit que especifica el modo de funcionamiento del procesador se encuentra en el registro de estado, Ea tiene activo el modo usuario. Seguidamente, el ciclo de aceptación cambia el registro de estado activando el modo privilegiado. Cuando más tarde se restituya al registro de estado el valor salvado Ea, que tiene activo el modo usuario, se pone el procesador en este modo. Por otro lado, al restituir el valor de contador de programa, se consigue que el procesador siga ejecutando a continuación del punto en el que el programa A fue interrumpido.

En la sección “3.10 Tratamiento de interrupciones” se detalla el tratamiento de las interrupciones realizado por el sistema operativo, dando soluciones al **anidamiento** de las mismas, fenómeno que se produce cuando se acepta una interrupción sin haberse completado el tratamiento de la anterior.

1.4. EL RELOJ

El término reloj se aplica a los computadores con tres acepciones diferentes, si bien relacionadas, como se muestra en la figura 1.10. Estas tres acepciones son las siguientes:

- Señal que gobierna el ritmo de ejecución de las instrucciones máquina (CLK).
- Generador de interrupciones periódicas o temporizador.
- Contador de fecha y hora, o reloj de tiempo real RTC (*Real Time Clock*).

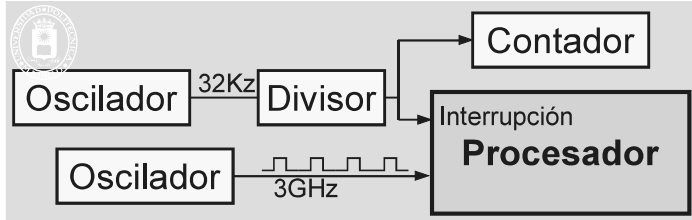


Figura 1.10 Relojes del computador.

El oscilador que gobierna las fases de ejecución de las instrucciones máquina se denomina reloj. Cuando se dice que un microprocesador es de 5 GHz, se está especificando que el oscilador que gobierna su ritmo de funcionamiento interno produce una onda cuadrada con una frecuencia de 5 GHz.

La señal producida por el oscilador anterior, o por otro oscilador, se divide mediante un divisor de frecuencia para generar una interrupción externa cada cierto intervalo de tiempo. Estas interrupciones, que se están produciendo constantemente, se denominan **interrupciones de reloj** o **tics**, dando lugar al segundo concepto de reloj. El objetivo de estas interrupciones es, como veremos más adelante, hacer que el sistema operativo entre a ejecutar de forma periódica. De esta manera, podrá evitar que un programa monopolice el uso del computador y podrá hacer que entren a ejecutarse programas en determinados instantes de tiempo. Estas interrupciones se producen con un periodo de entre 1 a 100 ms; por ejemplo, en la arquitectura PC clásica para MS-DOS el temporizador 8254 produce un tic cada 54,926138 ms.

La tercera acepción de reloj, denominada RTC, se aplica a un contador que permite conocer **la fecha y la hora**. Tomando como referencia un determinado instante (p. ej.: 0 horas del 1 de enero de 1990 (advertencia 1.3)) se puede calcular la hora y fecha en que estamos. Observe que este concepto de reloj es similar al del reloj electrónico de pulsera. En los computadores actuales esta cuenta se hace mediante un circuito dedicado que, además, está permanentemente alimentado, de forma que aunque se apague el computador se siga manteniendo el reloj en hora. En sistemas más antiguos el sistema operativo se encargaba de hacer esta cuenta, por lo que había que introducir la fecha y la hora cada vez que se arrancaba el computador. Por ejemplo, la arquitectura clásica PC incluye un RTC alimentado por una pequeña pila y gobernado por un cristal de 32.768 kHz, que almacena la hora con resolución de segundo.

Advertencia 1.3. En el caso de UNIX se toma como referencia las 0 horas del 1 de enero de 1970. Si se utiliza una palabra de 32 bits y se almacena la hora con resolución de segundo, el mayor número que se puede almacenar es el 2.147.483.647, que se corresponde a las 3h 14m y 7s de enero de 2038. Esto significa que, a partir de ese instante, el contador tomaría el valor 0 y la fecha volvería a ser el 1 de enero de 1970.

En algunos sistemas de tiempo real, o cuando se trata de monitorizar el sistema, es necesario tener una medida muy precisa del tiempo. En la arquitectura PC con el RTC se tiene una resolución de 1 segundo y de decenas de ms con los tics. Estos valores no son a veces suficientemente pequeños. Por ello, en el procesador Pentium se incluyó un registro de 64 bits que cuenta ciclos de la señal CLK y que se denomina TSC (*Time-Stamp Counter*). Este registro puede ser leído por las aplicaciones y suministra una gran resolución, pero hay que tener en cuenta que mide ciclos del oscilador y no tiempo. Un incremento de 4.000 en el TSC supone 2 μ s en un procesador de 2 GHz, mientras que supone 1 μ s en uno de 4 GHz.

1.5. JERARQUÍA DE MEMORIA

Dado que la memoria de alta velocidad tiene un precio elevado y un tamaño reducido, la memoria del computador se organiza en forma de una jerarquía, como la mostrada en la figura 1.11. En esta jerarquía se utilizan memorias permanentes de alta capacidad y baja velocidad, como son los discos, para almacenamiento permanente de la información, mientras que se emplean memorias de semiconductores de un tamaño relativamente reducido, pero de alta velocidad, para almacenar la información que se está utilizando en un momento determinado.

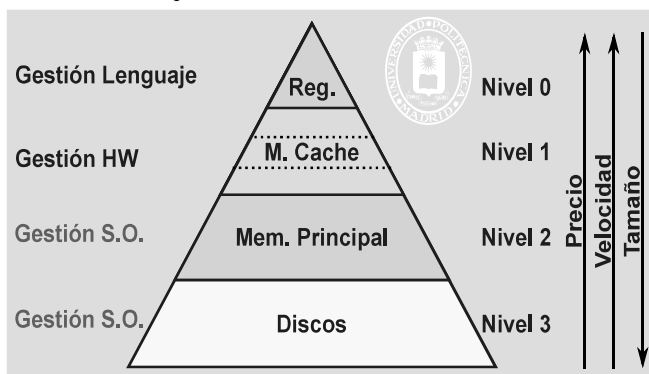


Figura 1.11 Jerarquía de memoria

El funcionamiento de la jerarquía de memoria exige hacer copias de información de los niveles más lentos a los niveles más rápidos, en los cuales es utilizada (p. ej.: cuando se desea ejecutar un programa hay que leer de disco el fichero ejecutable y almacenarlo en memoria principal, de donde se irá leyendo y ejecutando instrucción a instrucción por la unidad de control). Inversamente, cuando se modifica o crea la información en un nivel rápido, y se desea su permanencia, hay que enviarla al nivel inferior, por ejemplo, al nivel de disco o cinta.

Para entender bien el objetivo y funcionamiento de la jerarquía de memoria, es muy importante tener siempre presente tanto el orden de magnitud de los tiempos de acceso de cada tecnología de memoria como los tamaños típicos empleados en cada nivel de la jerarquía. La tabla 1.1 presenta algunos valores típicos.

Tabla 1.1 Valores típicos de la jerarquía de memoria.

Nivel de memoria	Capacidad	Tiempo de acceso	Tipo de acceso
Registros	64 a 1024 bytes	0,25 a 0,5 ns	Palabra
cache de memoria principal	8 KiB a 8 MiB	0,5 a 20 ns	Palabra
Memoria principal	128 MiB a 64 GiB	60 a 200 ns	Palabra
Disco electrónico SSD	128 GiB a 1 TiB	50 μ s (lectura)	Sector
Disco magnéticos	256 GiB a 4 TiB	5 a 30 ms	Sector

La gestión de la jerarquía de memoria es compleja, puesto que ha de tener en cuenta las copias de información que están en cada nivel y ha de realizar las transferencias de información a niveles más rápidos, así como las actualizaciones hacia los niveles permanentes. Una parte muy importante de esta gestión corre a cargo del sistema operativo, aunque, para hacerla correctamente, requiere de la ayuda del *hardware*. Por ello, se revisan en esta sección los conceptos más importantes de la jerarquía de memoria, para analizar más adelante su aplicación a la memoria virtual, de especial interés para nosotros, dado que su gestión la realiza el sistema operativo.

1.5.1. Memoria cache y memoria virtual

La explotación correcta de la jerarquía de memoria exige tener, en cada momento, la información adecuada en el nivel adecuado. Para ello, la información ha de moverse de nivel, esto es, ha de migrar de un nivel a otro. Esta migración puede ser **bajo demanda explícita** o puede ser **automática**. La primera alternativa exige que el programa solicite explícitamente el movimiento de la información, como ocurre, por ejemplo, con un programa editor, que va solicitando la parte del fichero que está editando el usuario en cada momento. La segunda alternativa consiste en hacer la migración transparente al programa, es decir, sin que éste tenga que ser consciente de que se produce. **La migración automática se utiliza en las memorias cache y en la memoria virtual.**

Memoria cache

El término cache deriva del verbo francés *cacher*, que significa ocultar, esconder. Con este término se quiere reflejar que la memoria cache no es visible al programa máquina, puesto que no está ubicada en el mapa de memoria. Se trata de una memoria de apoyo a la memoria principal que sirve para acelerar los accesos. La memoria cache alberga información recientemente utilizada, con la esperanza de que vuelva a ser empleada en un futuro próximo. Los aciertos sobre cache permiten atender al procesador más rápidamente que accediendo a la memoria principal.

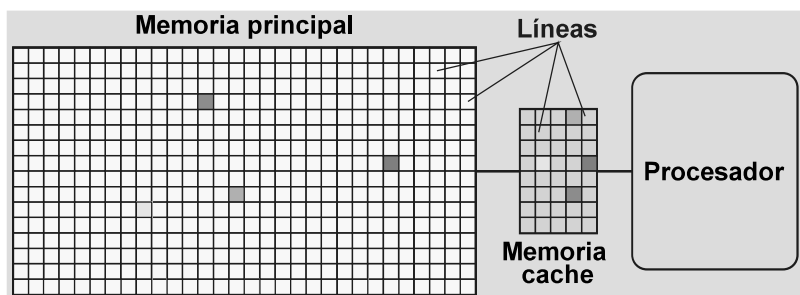


Figura 1.12 La memoria cache se interpone entre el procesador y la memoria principal para acelerar los accesos a esta última. La memoria cache almacena la información utilizada más recientemente con la esperanza de volver a utilizarla en un futuro próximo.

El bloque de información que se migra entre la memoria principal y la cache se denomina **línea** y está formado por varias palabras (valores típicos de línea son de 32 a 128 bytes). Toda la gestión de la cache necesaria para migrar líneas y para direccionar la información dentro de la cache se realiza por **hardware**, debido a la gran velocidad a la que debe funcionar. El tiempo de tratamiento de un fallo tiene que ser del orden del tiempo de acceso a la memoria lenta, es decir, de los 60 a 200 ns que se tarda en acceder a la memoria principal, puesto que el procesador se queda esperando a poder realizar el acceso solicitado.

En la actualidad, debido a la gran diferencia de velocidad entre los procesadores y las memorias principales, se utilizan tres niveles de cache, que se incluyen en el mismo chip que los procesadores.

Aunque la memoria cache es una memoria oculta, no nos podemos olvidar de su existencia, puesto que repercute fuertemente en las prestaciones de los sistemas. Plantear adecuadamente un problema para que genere pocos fallos de cache puede disminuir espectacularmente su tiempo de ejecución.

La memoria virtual versus memoria real

Una máquina con **memoria real** es una máquina convencional que **solamente utiliza memoria principal para soportar el mapa de memoria**. Por el contrario, una máquina con **memoria virtual soporta su mapa de memoria mediante dos niveles de la jerarquía de memoria: la memoria principal y una memoria de respaldo**, que suele ser una parte del disco que llamamos zona de intercambio o *swap*.

La memoria virtual es un mecanismo de migración automática, por lo que exige una gestión automática de la parte de la jerarquía de memoria formada por la memoria principal y el disco. Esta gestión la realiza el sistema operativo con ayuda de una unidad *hardware* de gestión de memoria, llamada **MMU** (*Memory Management Unit*) (ver sección “4.2.9 Unidad de gestión de memoria (MMU)”). Como muestra la figura 1.13, esta gestión incluye toda la memoria principal y la parte del disco que sirve de respaldo a la memoria virtual.

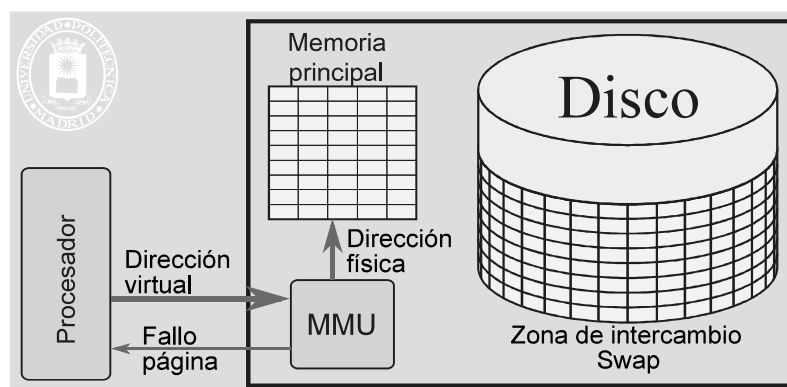


Figura 1.13 Fundamento de la memoria virtual.

Tanto el mapa de memoria (que denominaremos espacio virtual) como la memoria principal y el *swap* se dividen en páginas. Se denominan **páginas virtuales** a las páginas del mapa de memoria, **páginas de intercambio** a las páginas de *swap* y **marcos de página** a los espacios en los que se divide la memoria principal. Normalmente, cada marco de página puede albergar una página virtual cualquiera, sin ninguna restricción de direccionamiento.

Existe una estructura de información llamada **tabla de páginas** que contiene la ubicación de cada página virtual, es decir, el marco de página o la página de intercambio donde se encuentra. La MMU utiliza la tabla de páginas para traducir la dirección generada por el procesador a la dirección de memoria principal correspondiente. Si la dirección corresponde a una página que no está en memoria principal, la MMU genera una excepción de **fallo de página**, para que el sistema operativo se encargue de traer la página del disco a un marco de página. Será, por tanto, responsabilidad del sistema operativo la creación y mantenimiento de la tabla de páginas para que refleje en cada instante la ubicación real de las páginas.

La tabla de páginas puede tener un tamaño bastante grande y muy variable según las aplicaciones que se ejecuten, por lo que reside en memoria principal. Para acelerar el acceso a la información de la tabla de páginas la MMU utiliza la TLB (*Translation Look-aside buffer*). La TLB es una memoria asociativa muy rápida que almacena las parejas página-marco de las páginas a las que se ha accedido recientemente, de forma que en la mayoría de los casos la MMU no accederá a la memoria principal, al encontrar la información de traducción en la TLB. Cuando la MMU no encuentra la información de traducción en la TLB (lo que se denomina fallo de la TLB) debe acceder a memoria principal y sustituir una de las parejas página-marco de la misma por la nueva información. De esta forma, se va renovando la información de la TLB con los valores de interés en cada momento.

Operación en modo real

Como se ha indicado, el sistema operativo es el encargado de crear y mantener las tablas de páginas para que la MMU pueda hacer su trabajo de traducción. Ahora bien, cuando arranca el computador no existen tablas de páginas, por lo que la MMU no puede realizar su función. Para evitar este problema la MMU incorpora un modo de funcionamiento denominado real, en el cual se limita a presentar la dirección recibida del procesador a la memoria principal. En modo real el computador funciona, por tanto, como una máquina carente de memoria virtual.

- Gestor de comunicación y sincronización entre procesos. Ofrecer mecanismos para que los procesos puedan comunicarse y sincronizarse.
- Gestor de seguridad frente a ataques del exterior y protección interna. Este componente debe encargarse de realizar la identificación de los usuarios, de definir lo que pueden hacer cada uno de ellos con los recursos del sistema y de controlar el acceso a estos recursos.

Todos estos componentes ofrecen una serie de servicios a través de una interfaz de llamadas al sistema. Aunque no es muy frecuente, la figura 2.13 muestra que un sistema operativo puede incluir más de una interfaz de servicios, definiendo cada interfaz una máquina extendida propia. En la figura se han considerado las interfaces Windows y UNIX, interfaces que serán descritas a lo largo del presente libro. En este caso, los programas podrán elegir sobre qué máquina extendida quieren ejecutar, pero no podrán mezclar servicios de varias máquinas extendidas.

De igual forma, el sistema operativo puede incluir varios intérpretes de mandatos, unos textuales y otros gráficos, pudiendo el usuario elegir los que más le interesen, debiendo utilizar, en cada caso, los mandatos correspondientes.

En las secciones siguientes de este capítulo se van a describir, de forma muy breve, cada uno de los componentes anteriores.

2.5.1. Gestión de procesos

Como se indicó anteriormente, proceso es un programa en ejecución. De una forma más precisa, se puede definir el proceso como la unidad de procesamiento gestionada por el sistema operativo. No hay que confundir el concepto de programa con el concepto de proceso. Un programa no es más que un conjunto de instrucciones máquina, mientras que el proceso surge cuando un programa se pone en ejecución. Esto hace que varios procesos puedan ejecutar el mismo programa a la vez (por ejemplo, que varios usuarios estén ejecutando el mismo navegador). Para que un programa se pueda ejecutar tiene que estar preparado en un fichero ejecutable, que contiene el código y algunos datos iniciales (véase figura 4.27, página 166).

Dado que un computador está destinado a ejecutar programas, podemos decir que la misión más importante del sistema operativo es la generación de procesos y la gestión de los mismos.

Para ejecutar un programa éste ha de residir, junto con sus datos, en el mapa de memoria principal, tal y como muestra la figura 2.14. Se denomina **imagen de memoria** a la información que mantiene el proceso en el mapa de memoria.

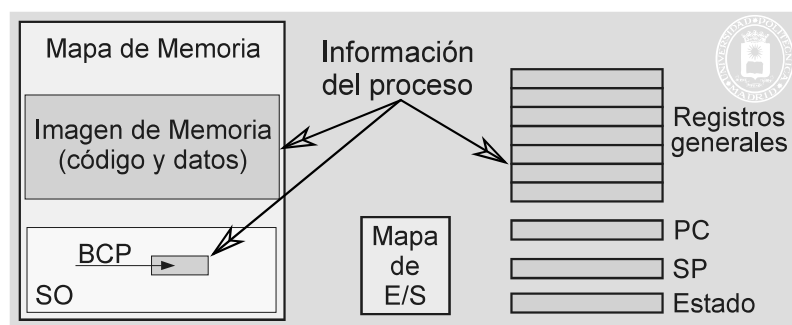


Figura 2.14 Elementos que constituyen un proceso.

Durante su ejecución, el proceso va modificando los contenidos de los registros del computador, es decir, va modificando el **estado del procesador**. También va modificando los datos que tiene en memoria, es decir, su imagen de memoria.

El sistema operativo mantiene, por cada proceso, una serie de estructuras de información, que permiten identificar las características de éste, así como los recursos que tiene asignados. Una parte muy importante de esta información está en el **bloque de control del proceso (BCP)**, que se estudiará con detalle en la sección “3.3.3 Información del bloque de control de proceso (BCP)”. El sistema operativo debe encargarse también de ofrecer una serie de servicios para la gestión de procesos y para su planificación, así como para gestionar los posibles interbloqueos que surgen cuando los procesos acceden a los mismos recursos.

Una buena parte de la información del proceso se obtiene del ejecutable, pero otra la produce el sistema operativo. A diferencia del ejecutable, que es permanente, la información del proceso es temporal y desaparece con el mismo. Decimos que el proceso es volátil mientras que el ejecutable perdura hasta que se borre el fichero.

En el capítulo “3 Procesos” se estudiarán en detalle los procesos y en el capítulo “6 Comunicación y sincronización de procesos” se estudiarán los interbloqueos y los mecanismos para manejarlos.

Servicios de procesos

El sistema operativo ofrece una serie de servicios que permiten definir la vida de un proceso, vida que consta de las siguientes fases: creación, ejecución y muerte del proceso, que se analizan seguidamente:

- **Crear un proceso.** El proceso es creado por el sistema operativo cuando así lo solicita otro proceso, que se convierte en el padre del nuevo. Existen dos modalidades básicas para crear un proceso en los sistemas operativos:

- ◆ Creación a partir de la imagen del proceso padre. En este caso, el proceso hijo es una copia exacta o clon del proceso padre. Esta variante es la que utiliza el servicio `fork` de UNIX.
- ◆ Creación a partir de un fichero ejecutable. Esta modalidad es la que se utiliza en el servicio `CreateProcess` de Windows.
- **Ejecutar un proceso.** Los procesos se pueden ejecutar de tres formas: *batch*, interactiva y segundo plano.
 - ◆ Un proceso que ejecuta en modo de **lotes** o *batch*, no está asociado a ningún terminal. Deberá tomar sus datos de entrada de ficheros y deberá depositar sus resultados en ficheros. Un ejemplo típico de un proceso *batch* es un proceso de nóminas, que parte del fichero de empleados y del fichero de los partes de trabajo para generar un fichero de órdenes bancarias de pago de nóminas.
 - ◆ Por el contrario, un proceso que ejecuta en modo **interactivo** está asociado a un terminal, por el que recibe la información del usuario y por el que contesta con los resultados. Un ejemplo típico de un proceso interactivo es un proceso de edición.
 - ◆ Los sistemas interactivos permiten lanzar procesos en segundo plano o *background*. Se trata de procesos similares a los de lotes, que no están asociados a ningún terminal.
- **Terminar la ejecución de un proceso.** Un proceso puede finalizar su ejecución por varias causas, entre las que se encuentran las siguientes:
 - ◆ El programa ha llegado a su final.
 - ◆ Se produce una condición de error en su ejecución, como división por cero o acceso de memoria no permitido.
 - ◆ Otro proceso o el usuario decide que ha de terminar y lo mata, por ejemplo, con el servicio `kill` de UNIX.
- **Cambiar el ejecutable de un proceso.** Algunos sistemas operativos incluyen un servicio que cambia, por otro, el ejecutable que está ejecutando un proceso. Observe que esta operación no consiste en crear un nuevo proceso que ejecuta ese nuevo ejecutable, se trata de sustituir el ejecutable que está ejecutando el proceso por un nuevo ejecutable que se trae del disco, manteniendo el mismo proceso. El servicio `exec` de UNIX realiza esta función.

2.5.2. Gestión de memoria

El componente del sistema operativo llamado gestor de memoria se encarga de:

- Asignar memoria a los procesos para crear su imagen de memoria.
- Proporcionar memoria a los procesos cuando la soliciten y liberarla cuando así lo requieran.
- Tratar los errores de acceso a memoria, evitando que unos procesos interfieran en la memoria de otros.
- Permitir que los procesos puedan compartir memoria entre ellos. De esta forma los procesos podrán comunicarse entre ellos.
- Gestionar la jerarquía de memoria y tratar los fallos de página en los sistemas con memoria virtual.

Servicios

Además de las funciones vistas anteriormente, el gestor de memoria ofrece los siguientes servicios:

- **Solicitar memoria.** Este servicio aumenta el espacio de la imagen de memoria del proceso. El sistema operativo satisfará la petición siempre y cuando cuente con los recursos necesarios para ello y no se exceda la cuota en caso de estar establecida. En general, el sistema operativo devuelve un apuntador con la dirección de la nueva memoria. El programa utilizará este nuevo espacio a través del mencionado apuntador, mediante direccionamientos relativos al mismo.
- **Liberar memoria.** Este servicio sirve para devolver trozos de la memoria del proceso. El sistema operativo recupera el recurso liberado y lo añade a sus listas de recursos libres, para su posterior reutilización. Este servicio y el de solicitar memoria son necesarios para los programas que requieren asignación dinámica de memoria, puesto que su imagen de memoria ha de crecer o decrecer de acuerdo a las necesidades de ejecución.
- **Compartir memoria.** Son los servicios de crear y liberar regiones de memoria compartidas, lo que permite que los procesos puedan comunicarse escribiendo y leyendo en ellas.

El gestor de memoria establece la imagen de memoria de los procesos. La figura 2.15 muestra algunas soluciones para sistemas reales y virtuales.

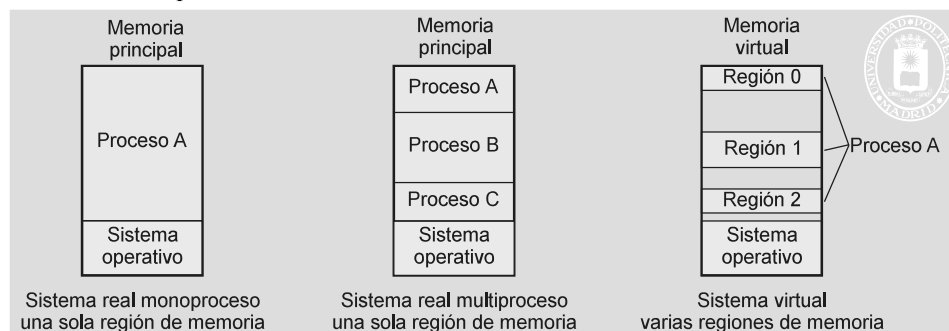


Figura 2.15 Distintas alternativas de asignación de memoria.

En el capítulo “4 Gestión de memoria” se estudiarán los conceptos relativos a la gestión de memoria, los servicios ofrecidos por el gestor de memoria y las técnicas de gestión de memoria.

2.5.3. Comunicación y sincronización entre procesos

Los procesos son entes independientes y aislados, puesto que, por razones de seguridad, no deben interferir unos con otros. Sin embargo, cuando se divide un trabajo complejo en varios procesos que cooperan entre sí para realizar dicho trabajo, es necesario que se comuniquen, para transmitirse datos y órdenes, y que se sincronicen en la ejecución de sus acciones. Por tanto, el sistema operativo debe incluir servicios de comunicación y sincronización entre procesos que, sin romper los esquemas de protección, han de permitir la cooperación entre ellos.

El sistema operativo ofrece una serie de mecanismos básicos de comunicación que permiten transferir cadenas de bytes, pero han de ser los procesos que se comunican entre sí los que han de interpretar las cadenas de bytes transferidas. En este sentido, se han de poner de acuerdo en la longitud de la información y en los tipos de datos utilizados. Dependiendo del servicio utilizado, la comunicación se limita a los procesos de una máquina (procesos locales) o puede involucrar a procesos de máquinas distintas (procesos remotos). La figura 2.16 muestra ambas situaciones.

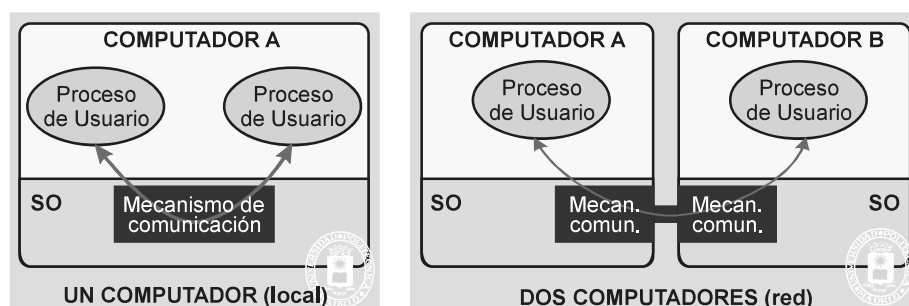


Figura 2.16 Comunicación entre procesos locales y remotos.

El sistema operativo ofrece también mecanismos que permiten que los procesos esperen (se bloqueen) y se despierten (continúen su ejecución) dependiendo de determinados eventos.

Servicios de comunicación y sincronización

Existen distintos mecanismos de comunicación (que en muchos casos también sirven para sincronizar), como son las tuberías o *pipes*, la memoria compartida y los *sockets*. Cada uno de estos mecanismos se puede utilizar a través de un conjunto de servicios propios. Estos mecanismos son entidades vivas, cuya vida presenta las siguientes fases: creación del mecanismo, utilización del mecanismo y destrucción del mecanismo. De acuerdo con esto, los servicios básicos de comunicación, que incluyen todos los mecanismos de comunicación, son los siguientes:

- **Crear.** Permite que el proceso solicite la creación del mecanismo. Ejemplo `pipe` de UNIX y `CreatePipe` de Windows.
- **Enviar o escribir.** Permite que el proceso emisor envíe información a otro proceso. Ejemplo `write` de UNIX y `WriteFile` de Windows.
- **Recibir o leer.** Permite que el proceso receptor reciba información de otro proceso. Ejemplo `read` de UNIX y `ReadFile` de Windows.
- **Destruir.** Permite que el proceso solicite el cierre o destrucción del mecanismo. Ejemplo `close` de UNIX y `CloseHandle` de Windows.

Por otro lado, la comunicación puede ser síncrona o asíncrona. En la comunicación **síncrona** los dos procesos han de ejecutar los servicios de comunicación al mismo tiempo, es decir, el emisor ha de estar ejecutando el servicio de enviar y el receptor ha de estar ejecutando el servicio de recibir. Normalmente, para que esto ocurra uno de ellos ha de esperar a que el otro llegue a la ejecución del correspondiente servicio (véase la figura 2.17).